

Conventions

This document describes the naming conventions, Discord server structure, and the division of responsibilities between the bot and the Discord UI. It is the canonical reference for anyone operating or extending the bot.

Multi-guild: The bot is fully stateless with respect to guilds. Every command derives all context exclusively from `interaction.guild` — no bot-level state is shared between servers. The same bot instance can serve any number of guilds independently and safely.

Role Naming Conventions

Role	Pattern	Example	Created by
Student role	<code>students</code>	<code>students</code>	Manual (once)
Group role	<code><year>_group_<n></code>	<code>2026_group_1</code>	Bot (<code>/start_semester</code>)
Alumni role	<code>Alumni</code>	<code>Alumni</code>	Bot (<code>/end_semester</code> , first run)

- `<year>` is the UTC calendar year in which `/start_semester` is run.
- `<n>` is a 1-based integer index up to the number of groups specified.
- **Group roles are year-scoped from creation.** A new set of `<year>_group_<n>` roles is created at the start of each semester. Roles from previous semesters are never renamed or deleted — members retain them and thereby keep read-only access to their archived channel automatically.
- The `students` role must be created **once manually** before the first semester (see One-Time Manual Setup).

Channel / Category Naming Conventions

Category	Name	Created by
Group text channels (active)	<code>group_text_channels</code>	Bot (<code>/start_semester</code>)
Group voice channels (active)	<code>group_voice_channels</code>	Bot (<code>/start_semester</code>)
Archived group text channels	<code>Archived_text_channels</code>	Bot (<code>/end_semester</code> , first run)

Category	Name	Created by
Shared student resources	<code>shared_channels</code>	Manual (once)

Individual channels inside `group_text_channels` and `group_voice_channels` are named after the group role they belong to (e.g. `2026_group_3`).

At end of semester, text channels are renamed to `<year>-group_<n>` (e.g. `2026-group_3`) and moved into `Archived_text_channels` with the following permission overwrites:

- `@everyone` — read denied
- `<year>-group_<n>` — read-only (no send, no reactions)
- Members with the **Administrator** permission bypass all overwrites automatically; no explicit admin overwrite is set.

Voice channels are deleted. Members already holding the `<year>-group_<n>` role automatically retain read-only access to the archived channel — no role changes are needed.

If a channel is not found in its expected category during `/end_semester`, the bot logs a warning and continues.

What the Bot Does

`/start_semester <number_of_groups: int>`

Requires **Administrator** permission. The bot must have `manage_roles` and `manage_guild` permissions; the command aborts with an error message if not.

1. Derives the current year from UTC (e.g. `2026`).
2. Creates `2026_group_1` ... `2026_group_n` roles — skips any that already exist with a warning.
3. Creates (or reuses) the `group_text_channels` and `group_voice_channels` categories, both hidden from `@everyone` — skips any channel that already exists inside the category with a warning.
4. Creates one private text channel and one private voice channel per group, named `<year>-group_<n>` and visible only to the matching role.
5. Creates or updates the Discord Onboarding prompt titled “*Which group are you in?*” so its options match the current group roles exactly. Each option assigns both the group role and the `students` role. Creates the prompt if it does not yet exist; otherwise replaces only its options, leaving all other prompts untouched.

Does not assign any roles to members — that is handled by Discord Onboarding.

/end_semester

Requires **Administrator** permission. The bot must have **manage_roles** and **manage_guild** permissions; the command aborts with an error message if not.

1. Derives the current year from UTC and scans all server roles for names matching `<year>_group_<n>` (current year only).
 2. For each matched role:
 - Assigns the **Alumni** role to every current member of that group (creates the role if it does not yet exist).
 - Looks up the group's text channel **inside group_text_channels only**, renames it to `<year>-group_<n>`, moves it to **Archived_text_channels**, and sets its overwrites to: deny @everyone, allow `<year>_group_<n>` (read-only). Logs a warning and continues if the channel is not found.
 - Looks up the group's voice channel **inside group_voice_channels only** and deletes it. Logs a warning and continues if the channel is not found.
 3. Removes the **students** role from **every member** who holds it.
 4. Deletes the now-empty **group_text_channels** and **group_voice_channels** categories.
 5. Removes the Discord Onboarding prompt “*Which group are you in?*” entirely (Discord does not allow prompts with zero options; it is re-created on the next `/start_semester`).
 6. **Does not rename, move, or delete** any `<year>_group_<n>` roles — members already hold the correctly-named role and gain read-only archive access automatically.
-

One-Time Manual Setup

The following must be configured **once** by an administrator before the bot is used for the first semester. The bot will not create or manage these.

1. Create the **students role**

In **Server Settings** → **Roles**, create a role named exactly **students**. Configure its permissions to allow access to the shared student resources you want all enrolled students to see.

2. Create the **shared_channels category and channels**

Create a category (suggested name: `shared_channels`) and populate it with channels that all students should have access to (e.g. `#announcements`, `#general`, `#resources`). Grant the **students** role view and send access on this category via Discord's permission system.

3. Configure Discord Onboarding (base setup)

In **Server Settings** → **Onboarding**:

- Enable Onboarding and add a **default role** assignment of `students` so every member joining receives it automatically.
 - The group-selection prompt (“*Which group are you in?*”) is fully managed by the bot — it will be created on the first `/start_semester` and removed at `/end_semester` automatically. No manual configuration of that prompt is needed.
-

How to Adopt This Bot (New Guild Setup)

Follow these steps the first time you deploy the bot to a new Discord server.

Step 1 — Invite the bot

Invite the bot to your server and ensure it has the following permissions:

- **Manage Roles**
- **Manage Guild** (required to read and edit Onboarding)
- **Manage Channels**
- **View Channels**

Important: The bot’s role in the role hierarchy must be **above** all `<year>_group_<n>` roles it will manage. Discord prevents bots from managing roles positioned above their own.

Step 2 — Create the `students` role

In **Server Settings** → **Roles**, create a role named exactly:

`students`

Give it whatever permissions your shared student channels require (typically read + send in `shared_channels`).

Step 3 — Create shared channels

Create a category for resources all students should access (e.g. `shared_channels`). Add channels such as `#announcements`, `#general`, and `#resources`. Use Discord’s permission system to grant the `students` role access to this category.

Step 4 — Enable Discord Onboarding

In **Server Settings** → **Onboarding**:

1. Enable Onboarding.

2. Add a **default role** assignment for **students** — every new member will automatically receive this role when they complete onboarding.
3. Complete any other onboarding questions your server needs (rules, channels to browse, etc.).

Do not create a group-selection prompt manually — the bot manages that one.

Step 5 — Run `/start_semester` at the start of each semester

`/start_semester number_of_groups: <n>`

The bot will:

- Derive the current UTC year (e.g. 2026).
- Create `2026_group_1 ... 2026_group_n` roles (skipped with a warning if any already exist).
- Create private text and voice channels named `2026_group_1 ... 2026_group_n` under dedicated categories.
- Create or update the “*Which group are you in?*” Onboarding prompt so students are assigned their group role (and **students**) automatically on join.

Step 6 — Run `/end_semester` at the end of each semester

`/end_semester`

The bot will:

- Scan roles matching `<year>_group_<n>` for the current UTC year.
- Assign the **Alumni** role to every member of each group.
- Archive each group’s text channel: renamed to `<year>-group_<n>`, moved to `Archived_text_channels`, restricted to the matching role (read-only). Members already hold the role — no channel-access role changes needed.
- Delete voice channels and the now-empty active categories.
- Remove the **students** role from all members.
- Remove the group-selection Onboarding prompt (re-created next semester).
- **Group roles are not deleted.** Members retain them and automatically keep read-only access to their archived channel.

Step 7 — Repeat each semester

At the start of the next semester, run `/start_semester` again. A new set of `<year>_group_<n>` roles is created for the new year. Old roles and archived channels from previous semesters are left untouched.

Semester Lifecycle Summary

Manual (once)	Bot	Discord Onboarding
Create students role		
Create shared_channels		
Configure Onboarding (students default role)		

— each semester —

```

/start_semester <n>
└ Derives current UTC year
└ Creates <year>_group_1...n roles
└ Creates private text + voice channels
  named <year>_group_1...n
└ Upserts onboarding prompt with
  <year>_group_1...n options (assigns
  students + <year>_group_x per option)
                                New member joins →
                                Onboarding assigns:
                                - students
                                - <year>_group_x

/end_semester
└ Scans roles matching <year>_group_<n>
  (current year only)
└ Assigns Alumni role to all group members
└ Archives text channels as <year>-group_<n>
  (read-only, restricted to <year>_group_<n>)
└ Deletes voice channels
└ Deletes active categories
└ Strips students role from all members
└ Removes onboarding group prompt
└ Group roles retained unchanged –
  members keep read-only archive access
  automatically

```